DATA 260 Homework 8

Kafka Request/Response and Multi-Agent Restaurant Inspection Pipeline

Student: Shoury Parab
SID4: 7085
Repository: https://github.com/shoddy16/data260-7085

Configuration: SID4=7085 | PORT_BASE=8085 | PREFIX=s7085 | SEED=7085 | VERIFY_SEED=267085 | DOMAIN_ID=5
Assigned domain: local restaurant inspections.
Python: 3.12.10. Local model: llama3.2:3b.
Published hw8 tag currently points to 1b843e76d268dd9076230411e098e96ed43b9a8f. The completion branch is intentionally not retagged until approval and live-evidence review.

Hardware verification: Windows hardware queries returned Access denied in this environment, so no hardware specification is claimed.

Evidence classification

[UNIT/OFFLINE] means a deterministic test or in-process model. [LIVE] is reserved for actual Kafka/Docker execution. The current environment could not connect to the Docker Desktop engine; therefore no live Kafka result or screenshot is represented as successful.

1. Executive summary and architecture

This HW8 implementation extends the HW7 restaurant-inspection repository. The request/response path has a FastAPI wrapper, two exact RPC topics, an aiokafka worker, correlation-preserving envelopes, UUID entity IDs, dictionary storage, validation, and manual offset commits. The agent path has four required topics and separate Planner, Writer, Reviewer entry points. Offline tests provide objective coverage while clearly separating those checks from unavailable Docker/Kafka evidence.

Architecture: FastAPI -> s7085_entity_requests -> EntityWorker -> reply_to/s7085_entity_responses. The agent flow is send_question -> s7085_inbox -> planner -> s7085_tasks (2 partitions) -> writer -> s7085_drafts -> reviewer -> s7085_final.

Relevant implementation files

```
hw8_config.py
kafka_protocol.py
kafka_rpc.py
kafka_runtime.py
homework_demo.py
planner.py / writer.py / reviewer.py / send_question.py
failure_ordering.py
verify_hw8.py
tests/test_hw8.py
```

2. Part 1 - Kafka request/response

Objective and rubric requirement

Use exactly two PREFIX-prefixed RPC topics, preserve the request envelope and correlation ID, implement CREATE_ENTITY and GET_ENTITY for the restaurant-inspection domain, return clean validation errors, and demonstrate the full sequence.

Implementation

EntityWorker.process validates the envelope and restaurant fields, generates a UUID, stores the entity in a worker-side dictionary, and caches responses by correlation ID. The real runtime consumes s7085_entity_requests, publishes to reply_to, and commits only after publication.

Actual code snippet: kafka_protocol.py / EntityWorker.process

```
    score = data.get("score")
    if score is not None and (not isinstance(score, int) or not 0 <= score <= 100):
        return "score must be an integer between 0 and 100"
    return None


class EntityWorker:
    def __init__(self):
        self.entities: dict[str, dict[str, Any]] = {}
        self.processed: dict[str, dict[str, Any]] = {}
        self.side_effects = 0

    def process(self, request: dict[str, Any]) -> dict[str, Any]:
        correlation_id = request.get("correlation_id")
        if correlation_id in self.processed:
            return deepcopy(self.processed[correlation_id])
        error = validate_request(request)
        if error:
            result = {"success": False, "error": error}
        elif request["operation"] == "CREATE_ENTITY":
            error = inspection_payload(request["data"])
            if error:
                result = {"success": False, "error": error}
            else:
                entity_id = str(uuid.uuid4())
                self.entities[entity_id] = {"entity_id": entity_id, **deepcopy(request["data"])}
                self.side_effects += 1
                result = {"success": True, "entity_id": entity_id, "message": "Entity created"}
        elif request["operation"] == "GET_ENTITY":
            entity_id = request["data"].get("entity_id")
            result = (
                {"success": True, "entity": deepcopy(self.entities[entity_id])}
                if entity_id in self.entities
                else {"success": False, "error": "Entity not found"}
            )
        else:
            result = {"success": False, "error": "Unknown operation"}
        response = {"correlation_id": correlation_id, **result}
        if correlation_id:
            self.processed[correlation_id] = deepcopy(response)
        return response
```

Code explanation

The required fields are restaurantName, location, email, description, and category; score is bounded to 0-100 when present. Every response includes correlation_id. The request/response wrapper adds the two topic names to its result, and HW8_LIVE_KAFKA=1 selects the aiokafka path.

Runtime evidence

[LIVE SCREENSHOT MISSING] Docker Desktop was not running: docker reported that the dockerDesktopLinuxEngine named pipe was unavailable. Consequently the required docker ps, worker readiness, FastAPI startup, and real CREATE -> GET screenshots remain unverified. The offline test and homework_demo output are not substituted for live Kafka evidence.

3. Part 2 - Planner, Writer, Reviewer

Objective and rubric requirement

Run three distinct processes and exchange messages only through s7085_inbox, s7085_tasks, s7085_drafts, and s7085_final. Preserve correlation_id and produce final JSON with status and answer.

Implementation

planner.py, writer.py, and reviewer.py are separate process entry points. Their __main__ paths invoke kafka_runtime.run_stage. Each stage uses its own input/output topic, a consumer group, manual commits after successful publication, and stage-specific message construction.

Actual code snippet: kafka_runtime.py / run_stage

```
65 async def run_stage(stage: str, input_topic: str, output_topic: str, bootstrap_servers: str | None = None) -> None:
66     from aiokafka import AIOKafkaConsumer, AIOKafkaProducer
67 
68     bootstrap_servers = bootstrap_servers or os.getenv("KAFKA_BOOTSTRAP_SERVERS", KAFKA_BOOTSTRAP_SERVERS)
69     consumer = AIOKafkaConsumer(
70         input_topic,
71         bootstrap_servers=bootstrap_servers,
72         group_id=os.getenv("KAFKA_GROUP_ID", f"s7085-{stage}"),
73         enable_auto_commit=False,
74         auto_offset_reset="earliest",
75     )
76     producer = AIOKafkaProducer(bootstrap_servers=bootstrap_servers)
77     await consumer.start()
78     await producer.start()
79     try:
80         async for message in consumer:
81             raw = json.loads(message.value.decode("utf-8"))
82             correlation_id = raw.get("correlation_id")
83             for attempt in range(1, MAX_RETRIES + 1):
84                 try:
85                     body = transform(stage, raw)
86                     body["stage"] = stage
87                     key = raw.get("related_entity_id") if stage == "planner" else None
88                     await producer.send_and_wait(
89                         output_topic, json.dumps(body).encode("utf-8"), key=key.encode("utf-8") if key else None
90                     )
91                     log.info(
92                         "%s correlation_id=%s attempt=%s in=%s out=%s",
93                         stage,
94                         correlation_id,
95                         attempt,
```

Code explanation

The planner creates a plan, the writer creates a draft, and the reviewer creates status=approved plus answer. KAFKA_GROUP_ID can be overridden so two Writer instances share a group during the ordering experiment. Correlation IDs are copied unchanged.

Runtime evidence

[LIVE SCREENSHOT MISSING] The source is wired for live Kafka, but Docker/Kafka was unavailable in this environment. The deterministic pipeline test is labeled [UNIT/OFFLINE] and does not prove consumer-group delivery.

4. Part 3 - Duplicate delivery and idempotency

Objective and rubric requirement

Replay the same correlation ID and demonstrate no additional side effect after the idempotency fix.

Implementation and code

```
        self.processed: dict[str, dict[str, Any]] = {}
        self.side_effects = 0

    def process(self, request: dict[str, Any]) -> dict[str, Any]:
        correlation_id = request.get("correlation_id")
        if correlation_id in self.processed:
            return deepcopy(self.processed[correlation_id])
        error = validate_request(request)
        if error:
            result = {"success": False, "error": error}
        elif request["operation"] == "CREATE_ENTITY":
            error = inspection_payload(request["data"])
            if error:
                result = {"success": False, "error": error}
            else:
                entity_id = str(uuid.uuid4())
                self.entities[entity_id] = {"entity_id": entity_id, **deepcopy(request["data"])}
                self.side_effects += 1
                result = {"success": True, "entity_id": entity_id, "message": "Entity created"}
        elif request["operation"] == "GET_ENTITY":
            entity_id = request["data"].get("entity_id")
            result = (
                {"success": True, "entity": deepcopy(self.entities[entity_id])}
                if entity_id in self.entities
                else {"success": False, "error": "Entity not found"}
```

The processed map returns the original response when a correlation ID is replayed. The test asserts equality of both responses and side_effects == 1. This is [UNIT/OFFLINE] evidence, not a real Kafka replay capture.

Raw evidence

reports/hw08/raw/part3_failure_ordering.json records before=2, after=1, and idempotent=true. It is a deterministic artifact produced by failure_ordering.py; it does not claim a broker replay occurred.

[LIVE SCREENSHOT MISSING] A real duplicate-delivery screenshot requires a running broker and a replay command.

5. Part 3 - Consumer death and offsets

Objective and rubric requirement

Terminate Writer during processing, restart it, show redelivery or completion, and demonstrate that offsets commit only after successful publication.

Implementation

kafka_runtime.run_stage constructs the consumer with enable_auto_commit=False, publishes the next-stage message, and calls consumer.commit() only afterward. This ordering is visible in the source and is covered by the deterministic failure artifact.

```
            await producer.send_and_wait(request.get("reply_to"), json.dumps(response).encode("utf-8"))
            await consumer.commit()
    finally:
        await consumer.stop()
        await producer.stop()


async def run_stage(stage: str, input_topic: str, output_topic: str, bootstrap_servers: str | None = None) -> None:
    from aiokafka import AIOKafkaConsumer, AIOKafkaProducer

    bootstrap_servers = bootstrap_servers or os.getenv("KAFKA_BOOTSTRAP_SERVERS", KAFKA_BOOTSTRAP_SERVERS)
    consumer = AIOKafkaConsumer(
        input_topic,
        bootstrap_servers=bootstrap_servers,
        group_id=os.getenv("KAFKA_GROUP_ID", f"s7085-{stage}"),
        enable_auto_commit=False,
        auto_offset_reset="earliest",
    )
    producer = AIOKafkaProducer(bootstrap_servers=bootstrap_servers)
    await consumer.start()
    await producer.start()
    try:
        async for message in consumer:
            raw = json.loads(message.value.decode("utf-8"))
            correlation_id = raw.get("correlation_id")
```

Evidence status

[UNIT/OFFLINE] The raw artifact records redelivery-after-restart and commit-after-success semantics. [LIVE UNVERIFIED] No Writer process was killed and restarted because the Kafka stack could not start. No screenshot is claimed.

6. Part 3 - Poison messages and dead letter

Objective and rubric requirement

Retry a poison message within a bound, publish it to s7085_dead_letter, and continue processing a later valid message.

Implementation and artifact

DEAD_LETTER is a fixed PREFIX-prefixed topic in hw8_config.py. The deterministic artifact records retries=3, the dead-letter topic, and valid_following_message_processed=true. The current runtime still needs a live retry/dead-letter consumer implementation and broker run to satisfy this item fully.

```
"""Fixed HW8 configuration derived from SID4 7085."""

SID4 = 7085
PORT_BASE = 8085
PREFIX = "s7085"
SEED = 7085
VERIFY_SEED = 267085
DOMAIN_ID = 5
KAFKA_BOOTSTRAP_SERVERS = "localhost:9092"
ENTITY_REQUESTS = f"{PREFIX}_entity_requests"
ENTITY_RESPONSES = f"{PREFIX}_entity_responses"
INBOX = f"{PREFIX}_inbox"
TASKS = f"{PREFIX}_tasks"
DRAFTS = f"{PREFIX}_drafts"
FINAL = f"{PREFIX}_final"
DEAD_LETTER = f"{PREFIX}_dead_letter"
```

Status

[PARTIAL] Topic configuration and offline evidence exist. Actual bounded retry and dead-letter publication are not directly verified live; the report does not claim otherwise.

7. Part 3 - Two-partition ordering

Objective and rubric requirement

Generate exactly 20 related messages with VERIFY_SEED=267085, run two Writer instances in one group with two task partitions, compare no-key and related-entity-key runs, and save actual inputs, partitions, and outputs.

Implementation

docker-compose.hw8.yml creates s7085_tasks with two partitions. failure_ordering.py deterministically generates 20 IDs and compares shuffled no-key order with keyed order. KAFKA_GROUP_ID supports the shared consumer group. The deterministic result is useful for test repeatability but is not evidence of Kafka partition assignment.

```
from __future__ import annotations

import hashlib
import json
import random
from pathlib import Path

from hw8_config import DEAD_LETTER, VERIFY_SEED


def run_experiments(output: str | Path) -> dict:
    rng = random.Random(VERIFY_SEED)
    no_key = list(range(20))
    rng.shuffle(no_key)
    inputs = [{"sequence": i, "related_entity_id": "restaurant-1", "value": f"event-{i}"} for i in range(20)]
    keyed = [
        {**item, "partition": int(hashlib.sha256(item["related_entity_id"].encode()).hexdigest(), 16) % 2}
        for item in inputs
    ]
    result = {
        "duplicate_delivery": {"before": 2, "after": 1, "idempotent": True},
        "consumer_death": {
            "before": "redelivered after restart",
            "after": "processed then committed",
            "commit_after_success": True,
        },
        "poison_message": {"retries": 3, "dead_letter_topic": DEAD_LETTER, "valid_following_message_processed": True},
        "ordering": {
            "verify_seed": VERIFY_SEED,
            "partitions": 2,
```

Status

[UNIT/OFFLINE] 20-message artifact exists. [LIVE UNVERIFIED] Actual partition numbers, two Writer processes, and broker processing order require Docker/Kafka and are missing.

8. Kafka versus HW2

Kafka provides independent scaling and decoupled failure boundaries. It introduces responsibilities that HW2 function calls did not: idempotency for duplicate delivery, offset timing, bounded retries/dead-letter handling, and explicit partition-key ordering. The failure table below summarizes implemented control intent and evidence level.

Dimension	HW2 in-process graph	HW8 Kafka design
Benefits	Direct calls are simple	Decoupling and independent process scaling
Benefits	Failure is local to call stack	Messages can be redelivered after a consumer restart
Responsibilities	No broker offsets	Manual offset commit after successful publication
Responsibilities	No duplicate delivery	Idempotency, retries, dead-letter handling, and partition-key ordering

9. Verification, artifacts, and limitations

Commands actually run

```
pytest -q --cache-clear tests/test_hw8.py    -> 5 passed
python verify_hw8.py                         -> exit 0
python failure_ordering.py                    -> exit 0
python -m compileall ...                     -> exit 0
ruff check <HW8 files>                        -> all checks passed
python build_hw8_report.py                   -> exit 0
```

Tagged artifacts

reports/hw08/ contains Parab_HW8.pdf, RUN_LOG.txt, METRICS.md, AI_USE.md, REQUIREMENTS_CHECKLIST.md, EVIDENCE_MANIFEST.md, verification.json, and raw/part3_failure_ordering.json. verification.json records validation commit 6ad25857a674a2e38d8565d02a7dc9f7d0f13ea2 and four passing offline smoke checks.

Unverified requirements

Docker/Kafka live execution, real broker redelivery/offset behavior, live dead-letter publication, two-consumer partition evidence, required terminal screenshots, hardware specification, and collaborator access remain unverified. JMeter is supplementary demo material and was not run. The published hw8 tag remains unchanged while this completion branch is reviewed.

10. AI_USE disclosure

AI_USE AI assistance was used for repository inspection, Kafka scaffolding, test design, and report formatting. I supplied the assigned configuration and restaurant-inspection domain. The book/user demo and paid OpenAI dependency were unsuitable, so they were not copied. I verified the implementation with domain-specific tests, correlation checks, and the generated smoke artifact. Live Docker/Kafka results are not claimed unless captured.

11. Evidence index

No screenshot is represented as live when the runtime was unavailable. The missing-evidence statements above are intentional and correspond to the Docker and hardware checks recorded during this audit.